

DHWANI

Outbound AI Voice Agent Simulation Suite

SYSTEM ARCHITECTURE & PIPELINE REPORT

Project Name:	Dhwani (AI Voice Agent Simulator)
Document Type:	System Architecture, Design & Data Pipeline Reference
Author:	Antigravity AI Assistant
Target Audience:	Evaluators, Engineers, and Project Reviewers
Status:	Operational Prototype Successfully Deployed
Core Services:	FastAPI, React.js, Web Speech API, Microsoft Edge-TTS
Supported LLMs:	Google Gemini 3.5 Flash, Groq Cloud (Llama 3.1)

This document contains a comprehensive breakdown of the voice synthesis, large language model integration, browser-native speech recognition, and system design to ensure readiness for project evaluations and technical walkthroughs.

1. EXECUTIVE SUMMARY & THE SOLE PURPOSE OF DHWANI

Dhwani is a real-time, low-latency conversational simulation suite designed for testing, prototyping, and evaluating outbound AI Voice Agents. By establishing a duplex WebSocket connection between browser-native APIs and a backend Python service, Dhwani mimics an actual voice call experience directly inside a standard web browser.

The Sole Purpose of the Project:

The primary purpose of Dhwani is to solve the high barrier of entry, high cost, and slow iteration cycle associated with developing voice-based LLM agents. Normally, testing an outbound conversational voice agent requires setting up expensive telephony connections, renting virtual numbers, configuring SIP trunks (via Twilio or Telnyx), and routing audio streams into paid orchestrators like Retell AI or Vapi. This creates immense friction during early engineering phases.

Dhwani bridges this gap by acting as a zero-cost local simulator. It bypasses telephony networks entirely, substituting them with the browser microphone, browser-native Speech-to-Text, and a high-performance backend orchestrator. Developers can interactively test, debug, and demo their voice scripts, system prompts, latency bottlenecks, and voice accents instantly and for free.

Key Project Objectives:

- **Interactive Prototyping:** Allows engineers to tune outbound calling agents (e.g. appointment confirmers, lead qualifiers) interactively prior to production deployment.
- **Accurate Prompt Engineering:** Test how LLM models adhere to strict instructions (like 'keep responses under two sentences' or 'speak empathetically') under voice-turn conditions.
- **Voice Accent Customization:** Evaluate multiple synthetic voices and accents (US, UK, French, German) to match target demographics without altering configuration code.
- **Bandwidth & Cost Optimization:** Utilizes browser-based Speech-to-Text, sending only light text strings to the server, preserving network bandwidth and avoiding expensive audio upload processing costs.

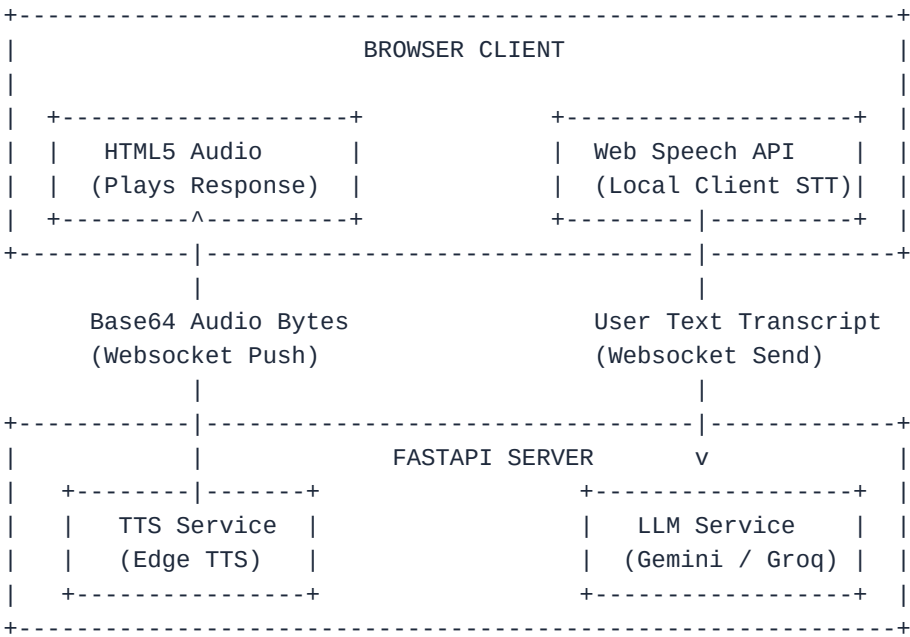
2. HIGH-LEVEL ARCHITECTURE & TECH STACK

Dhwani utilizes a modular, client-server architecture. It operates via stateful WebSocket sessions to maintain context and minimize audio transmission overhead. Below is the technical breakdown:

Technology Stack Elements:

- **Frontend UI (React + Vite):** A highly responsive glassmorphic dashboard built using React. It coordinates state transitions, renders the simulated calling layout, and tracks session timers.
- **Backend Server (FastAPI):** An asynchronous, high-concurrency Python web framework that handles JSON messages and coordinates streaming logic over WebSocket.
- **Speech-to-Text (STT):** Browser-native Web Speech API (SpeechRecognition). It performs local transcription directly inside the client's browser, eliminating raw audio upload latency.
- **Text-to-Speech (TTS):** Edge-TTS package, wrapping Microsoft's Azure Cognitive Speech Synthesis. Generates highly realistic, expressive neural voices without requiring paid API keys.
- **LLM Brain Options:** Google Gemini API (Gemini 3.5 Flash) and Groq Cloud (Llama 3.1). These providers act as the natural language generation engines.

Conceptual Pipeline Topology Diagram:



3. THE END-TO-END PIPELINE EXECUTION FLOW

A conversational session consists of two distinct parts: a setup sequence, followed by an alternating, state-controlled turn-taking loop between the user and the agent.

Step 1: Session Handshake & Configuration

The browser opens a stateful connection to the backend via 'ws://localhost:8000/ws/call'. Upon connection, the frontend sends a 'start_call' JSON payload containing system configurations (system instructions, agent voice accent selection, LLM provider settings, and opening greeting).

Step 2: Greeting Synthesizing & Playback

The FastAPI server initializes the LLM wrapper and invokes the Microsoft Edge-TTS library to convert the pre-set greeting text into speech. The synthesized audio bytes are returned as a Base64-encoded MP3 stream. The browser client receives the payload, decodes the audio, programmatically mutes the microphone to avoid feedback echo, and plays it. Once the audio completes, the mic is unmuted to listen to the user.

Step 3: User Speech Capture & Local Transcription

The user talks into their microphone. The Web Speech API listens and outputs real-time, interim speech transcripts. Once the user stops talking, the browser's SpeechRecognition engine fires the final callback, obtaining the final transcribed text. Doing this locally saves significant network overhead and cloud costs.

Step 4: AI Reply Generation

The final transcript is sent to the backend FastAPI server via the active WebSocket. The server updates the conversational session state to 'thinking' and logs the prompt history. It queries the user's selected LLM provider (Gemini or Groq) using the updated chat history and custom system instruction parameters.

Step 5: Voice Generation & Playback Loop

The generated text response is immediately sent to the Edge-TTS engine to synthesize the speech audio. The server pushes a JSON message containing the AI reply text and the Base64 audio stream. The browser plays the audio (programmatically muting the microphone during playback), displays the text on the screen, and reactivates speech recognition when the playback concludes, ready for the next user turn.

4. FILE-BY-FILE CODEBASE WALKTHROUGH

Below is a structural index of the project's files, outlining the role that each script plays in the system:

Backend Components (backend/app/):

- **app/main.py (Application Router & Websocket Controller)**: The FastAPI app instance, provides configuration APIs for frontend setup, and handles the websocket endpoint '/ws/call'. It acts as the orchestrator, controlling turn-taking by handling messages like 'start_call', 'user_speech', and 'hang_up'.
- **app/config.py (Settings Loader)**: Implements a Settings class that inherits from Pydantic's BaseSettings. It loads variables from the local '.env' file, providing credentials (GEMINI_API_KEY, GROQ_API_KEY) and defaults (LLM provider, model type, port) across the backend modules.
- **app/llm.py (LLM Provider Interface)**: LLMService. Integrates both Google Generative AI (using the new gemini-3.5-flash model) and Groq Client libraries. Standardizes input history shapes and queries the requested model. It receives chat histories and applies the custom system prompts dynamically.
- **app/tts.py (Voice Synthesis Engine)**: TSService. Integrates with the 'edge-tts' package. It uses Microsoft's neural voices, streams audio chunk-by-chunk locally, and collects raw MP3 bytes. It also defines a static list of curated voices (US, UK, French, German, Spanish).

Frontend Components (frontend/src/):

- **src/App.jsx (Client Controller & Interface Layout)**: The main entry point and logic of the UI. It instantiates the Web Speech API's SpeechRecognition controller, manages websocket open/close connections, plays incoming base64 MP3 files using HTML5 Audio elements, and coordinates microphone activation states to prevent speech loopbacks (echo).
- **src/index.css & App.css (Styling & Visual Design)**: Implements a modern glassmorphic dashboard styling. Features dark backgrounds, glowing circular voice-state visualizers, sliding settings drawers, and dynamic CSS animations representing audio waveforms.

5. TROUBLESHOOTING & VIVA/EVALUATION REFERENCE

Use this section to prepare for technical reviews. It explains design justifications and core FAQs.

Core Design Trade-offs & Decisions:

- **Client-Side STT vs Server-Side STT** Client-Side STT (browser Web Speech API) is free, requires no server compute, and is exceptionally low-latency. However, it requires modern engines (Chrome/Edge) and cannot record raw audio on the backend easily.
- **Edge TTS vs Premium Cloud TTS** Edge TTS generates highly realistic natural voices for free. It is ideal for simulation. For a full production system, paid APIs like ElevenLabs offer lower latency tuning and custom voice cloning.
- **WebSocket vs REST HTTP API** WebSockets keep a stateful TCP tunnel open. This maintains the LLM chat history in server memory during the call. It removes connection overhead on each turn, bringing average latency down.

Frequently Asked Evaluation Questions (Q&A):

- **Q1: How does Dhvani prevent the feedback loop (microphone picking up agent's speech)?** We use a WebSpeechRecognition engine. It listens to the HTML5 Audio 'onended' event, and only reactivates the microphone once the audio playback has completely stopped.
- **Q2: Why does the project use WebSockets instead of HTTP request-response?** WebSockets enable full-duplex communication, letting the server push state changes instantly (e.g. telling the frontend when the backend starts 'thinking' or 'speaking').
- **Q3: How are custom LLM credentials handled in the application?** Credentials are loaded from the '.env' file. However, users can also input custom keys in the frontend settings panel. The frontend passes these custom credentials in the setup message, allowing developers to test with their own quotas.
- **Q4: What happens if the network drops mid-call?** The React frontend detects a WebSocketDisconnect event and gracefully cleans up resources. The React frontend updates its status to 'disconnected' and resets the call states, preventing stale audio loops.